

Question Bank — Week 5

Linked lists (Lecture 05) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Questions: *week05-questions.md*. Commit to your own answer before reading one here — and for linked lists, draw it.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	b	M13	a	M19	c
M02	c	M08	d	M14	b	M20	d
M03	b	M09	a	M15	c	M21	a
M04	d	M10	c	M16	d	M22	b
M05	a	M11	b	M17	a		
M06	c	M12	d	M18	b		

W5-M01 — a. The order is in the links themselves; there are no indices (b) and no separate order (d).

W5-M02 — c. Follow next i times from head: $O(i)$, $O(n)$ in the worst case.

W5-M03 — b. Two reference assignments, independent of n . “Amortised” (c) describes the dynamic array’s append, not this.

W5-M04 — d. It must walk to the last node first. A tail reference makes it $O(1)$.

W5-M05 — a. Front insertion: two references here; every element shifts in an array.

W5-M06 — c. Indexing: address arithmetic in an array, a walk in a list. `len` (d) is $O(1)$ in both — because the list keeps `_size`.

W5-M07 — b. After `self.head = new`, `new.next = self.head` makes the node point at itself, and the old first node is no longer referenced by anything. Python evaluates each assignment’s right-hand side first (a), but that is not the reason for the order of two separate statements.

W5-M08 — d. Only the predecessor holds the reference that must be redirected around the removed node.

W5-M09 — a. The head has no predecessor, so it is removed by moving head instead.

W5-M10 — c. Each `ll[i]` walks i steps from the head: $0 + 1 + \dots + (n - 1)$. Use `for value in ll:` for one $O(n)$ walk.

W5-M11 — b. A stored count, updated by `push_front`, `append`, `insert_at`, `pop_front` and `remove`. Forget `one`, and `len` lies.

W5-M12 — d. Three references — `prev`, `node` and the saved next — whatever n is.

W5-M13 — a. One 8-byte reference per slot; the node is six times larger.

W5-M14 — b. With `prev` in each node, the node itself gives you its predecessor. Indexing (a) and search (c) are still $O(n)$.

W5-M15 — c. With a permanent node before the first real one, removing the first real node looks exactly like removing any other.

W5-M16 — d. After removing the last node, `_tail` must move to its predecessor, and only a walk from the head finds it in a singly linked list.

W5-M17 — a. Blocks give array-like memory efficiency; the links give $O(1)$ at both ends.

W5-M18 — b. It is the standard off-by-one: `node.next is not None` is the right test only when you *want* to stop on the last node (to append after it).

W5-M19 — c. `c.next` is `b`, and `b.next` is `a`, whose value is 1.

W5-M20 — d. Unreachable objects are reclaimed automatically. `del (b)` only removes a *name*.

W5-M21 — a. The node at $i - 1$ is the one whose next must change. Index 0 is `push_front`.

W5-M22 — b. `ChainingHashMap` (Week 13) stores an `Array` of chains of `Entry` nodes.

Part B — Short answer and essay

W5-E1 (4)

Operation	Dynamic array	Singly linked list
read element i	$O(1)$	$O(n)$
insert / remove at the front	$O(n)$	$O(1)$
append at the end	amortised $O(1)$	$O(n)$, or $O(1)$ with a tail
insert next to a node you hold	$O(n)$	$O(1)$

- **Choose the array** when you index by position, iterate a lot, or care about memory — e.g. storing measurements you read by index. It is compact and cache-friendly.
- **Choose the list** when you insert and remove at the front, or splice at positions you already hold — e.g. a stack of undo actions, the chains of a hash table. No copying, ever.

W5-E2 (4)

- Step 1: `new.next = self.head` — the new node points at the current first node.

- Step 2: `self.head = new` — the list now starts at the new node. Then `_size += 1`. Two assignments: $O(1)$.
- Diagram: `new node → old first node → ... → None`, with `head` moving from the old first node to the new one.
- **Reversed order:** `self.head = new` first, then `new.next = self.head` sets `new.next = new` — a node pointing at itself. Nothing refers to the old first node any more, so the rest of the list is lost, and iterating loops for ever over the one node.

W5-E3 (3)

- Array indexing is address arithmetic, $O(1)$. List indexing must follow `next` from `head` i times, $O(i)$ — the same interface hides a different cost.
- In the loop, each `ll[i]` starts again from the head: $0 + 1 + \dots + (n - 1) = O(n^2)$ for what should be one pass.
- Write `for value in ll:` — `__iter__` keeps its place and walks once: $O(n)$.

W5-E4 (4)

- **Benefit:** append links after `_tail` and moves `_tail`: $O(1)$ instead of a walk.
- **Cost:** one more reference to keep correct in every method that can change the last node: `push_front` and append on an **empty** list (the new node is also the tail), `insert_at` at the end, `pop_front` of the **only** node, remove of the last node, and reverse (head and tail swap).
- **Removing the last node** is still $O(n)$: `_tail` must move to the node before it, and in a singly linked list only a walk from the head finds that node.

W5-E5 (3)

- **Doubly linked:** each node also references its predecessor. Removing a node you hold, and removing from the end, become $O(1)$; the list can be walked backwards. Cost: one more reference per node, and every insertion and removal updates twice as many links.
- **Sentinel:** a permanent node before the first real one, so every real node has a predecessor and there is no special case for the head (or for an empty list). Cost: one extra node, and every method must remember that the first real node is `sentinel.next`.

Part C — Trace the code

W5-T1

3 2 1

Each value is pushed on the **front**, so the last one pushed comes first — the behaviour of a stack (Week 6).

W5-T2

1 3 4 None

`a.next = a.next.next` makes node 1 skip node 2 — exactly how `remove` works.

W5-T3. 5 nodes → **2**; 6 nodes → **3**; 1 node → **0**; None → **0**. It moves two nodes at a time and counts the moves: it returns $n // 2$, half the length rounded down, in one pass — the “fast reference” of W5-C1.

W5-T4.

After step	prev	node	Links
start	None	1	1 → 2 → 3 → None
1	1	2	1 → None; 2 → 3 → None
2	2	3	2 → 1 → None; 3 → None
3	3	None	3 → 2 → 1 → None

Then `self.head = prev = 3`.

W5-T5

b True

`x.next = y` closes a loop: $a \rightarrow b \rightarrow a \rightarrow b \dots$ — a **cycle**. Iterating it would never end; W5-C2 detects it.

Part D — List state

W5-S1.

<code>LinkedList([1, 2, 3])</code>	<code>1 → 2 → 3</code>	
<code>push_front(0)</code>	<code>0 → 1 → 2 → 3</code>	
<code>insert_at(2, 9)</code>	<code>0 → 1 → 9 → 2 → 3</code>	
<code>pop_front()</code>	<code>1 → 9 → 2 → 3</code>	(returns 0)
<code>remove(2)</code>	<code>1 → 9 → 3</code>	(returns True)
<code>append(5)</code>	<code>1 → 9 → 3 → 5</code>	

`len(ll)` is **4**; `ll.find(3)` is **2**.

W5-S2. `head` points at the new node 5, and then `5.next = self.head` makes it point at **itself**. Nodes 3 and 7 are unreachable (and collected).

<code>head → 5 → 5 → 5 → ...</code>	(5.next is node 5 itself)
<code>3 → 7 → None</code>	(unreachable: lost)

`list(ll)` never finishes: `__iter__` yields 5, 5, 5, ... for ever. (Press Ctrl+C.)

W5-S3. 7 is not at the head, so walk with `prev`:

start:	<code>head, prev → 3 → 7 → 1 → 9 → None</code>	<code>prev.next.value</code> is 7: match
splice:	<code>prev.next = prev.next.next</code>	
result:	<code>head → 3 → 1 → 9 → None</code>	node 7: unreachable

`_size` goes from 4 to 3, and `remove` returns True.

Part E — Complexity analysis

W5-K1 — $\Theta(n^2)$. The i -th append walks past the i nodes already there: $0 + 1 + \dots + (n - 1) = n(n - 1)/2$. **In** $\Theta(n)$: `push_front` every value ($O(1)$ each) and then `reverse()` once ($O(n)$) — or keep a tail reference.

W5-K2 — $\Theta(n^2)$. The `for` loop is one walk, n iterations; each `find(value)` walks from the head to that value's position: $1 + 2 + \dots + n$ comparisons in total.

W5-K3. Make the **head** the front (dequeue there) and the **tail** the back (enqueue there). Enqueue: link after `_tail`, $O(1)$. Dequeue: `pop_front`, $O(1)$. The other way round, dequeue would have to remove the **last** node, which is $O(n)$ in a singly linked list (W5-M16).

Part F — Find and fix the bug

W5-B1. `while node.next is not None` stops **on** the last node without checking it: `find` of the last value returns `-1`. On an empty list, `node` is `None` and `node.next` raises `AttributeError`. **Fix**: `while node is not None:`.

W5-B2. It never checks the **head** itself: `remove` of the first value returns `False` and leaves it in place. **Fix**: handle the head first —

```
if self.head is not None and self.head.value == value:
    self.head = self.head.next
    self._size -= 1
    return True
```

— or use a sentinel.

W5-B3. Two bugs: on an empty list `self.head.value` raises `AttributeError` instead of the promised `IndexError`; and `_size` is never decreased, so `len` becomes wrong. **Fix**: `if self.head is None: raise IndexError(...)` first, and `self._size -= 1`.

W5-B4. `node = node.next` runs **after** `node.next = prev`, so it moves to `prev` — backwards — instead of on. On $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, the first step sets `1.next = None` and then `node = None`: the loop ends, `head` becomes node 1, and the list is just `[1]`; nodes 2, 3, 4 are lost. **Fix**: save the next node first:

```
while node is not None:
    following = node.next
    node.next = prev
    prev = node
    node = following
```

Part G — Write the code

W5-C1 — slow and fast references.

```
def middle_value(head):
    if head is None:
        raise ValueError("empty chain")
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next           # one step
        fast = fast.next.next      # two steps
    return slow.value
```

When fast reaches the end, slow has gone half as far.

W5-C2 — Floyd's "tortoise and hare".

```
def has_cycle(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:           # the fast one lapped the slow one
            return True
    return False
```

Why they meet: without a cycle, fast reaches None. With one, both references end up inside the loop; each step the distance from fast to slow along the loop shrinks by exactly one, so it reaches 0 within one lap. Note `is`, not `==`: the question is whether they are the **same node**.

W5-C3 — a sentinel makes the head no special case.

```
def merge_sorted_chains(first, second):
    dummy = Node(None)
    tail = dummy
    while first is not None and second is not None:
        if first.value <= second.value:  # <= keeps it stable
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first if first is not None else second
    return dummy.next
```

The final line links the remaining chain in one step — no loop needed, unlike an array merge. $O(n + m)$ time, $O(1)$ extra space.

W5-C4

```
def remove_duplicates_sorted(head):
    node = head
    while node is not None and node.next is not None:
        if node.next.value == node.value:
            node.next = node.next.next    # skip the duplicate; stay on node
        else:
            node = node.next
    return head
```

Staying on node after a removal matters: the new `node.next` may be another duplicate.

W5-C5 — two references k apart.

```
def kth_from_end(head, k):
    if k < 1:
        raise IndexError(k)
    lead = head
    for _ in range(k):                # move lead k nodes ahead
        if lead is None:
            raise IndexError(k)
        lead = lead.next
    trail = head
    while lead is not None:           # move both until lead falls off the end
        lead = lead.next
        trail = trail.next
    return trail.value
```

When `lead` is `None`, `trail` is exactly k nodes before the end.